

MBA EM **ENGENHARIA DE SOFTWARE**

Paradigmas de Desenvolvimento de Software

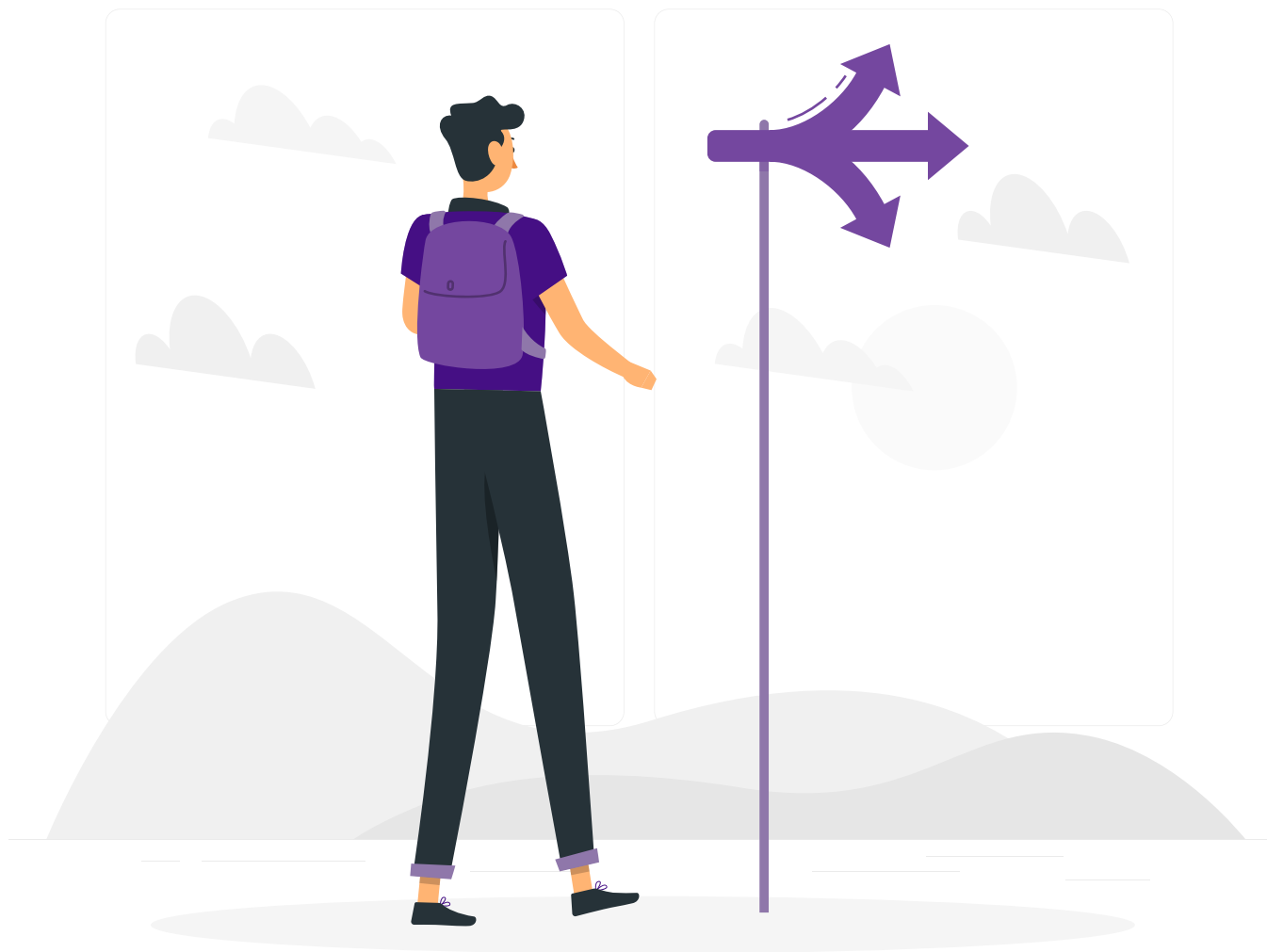
Prof. Helder Prado Santos

La responsabilidad por la idoneidad, originalidad y licitud de los contenidos didácticos presentados es del profesor.

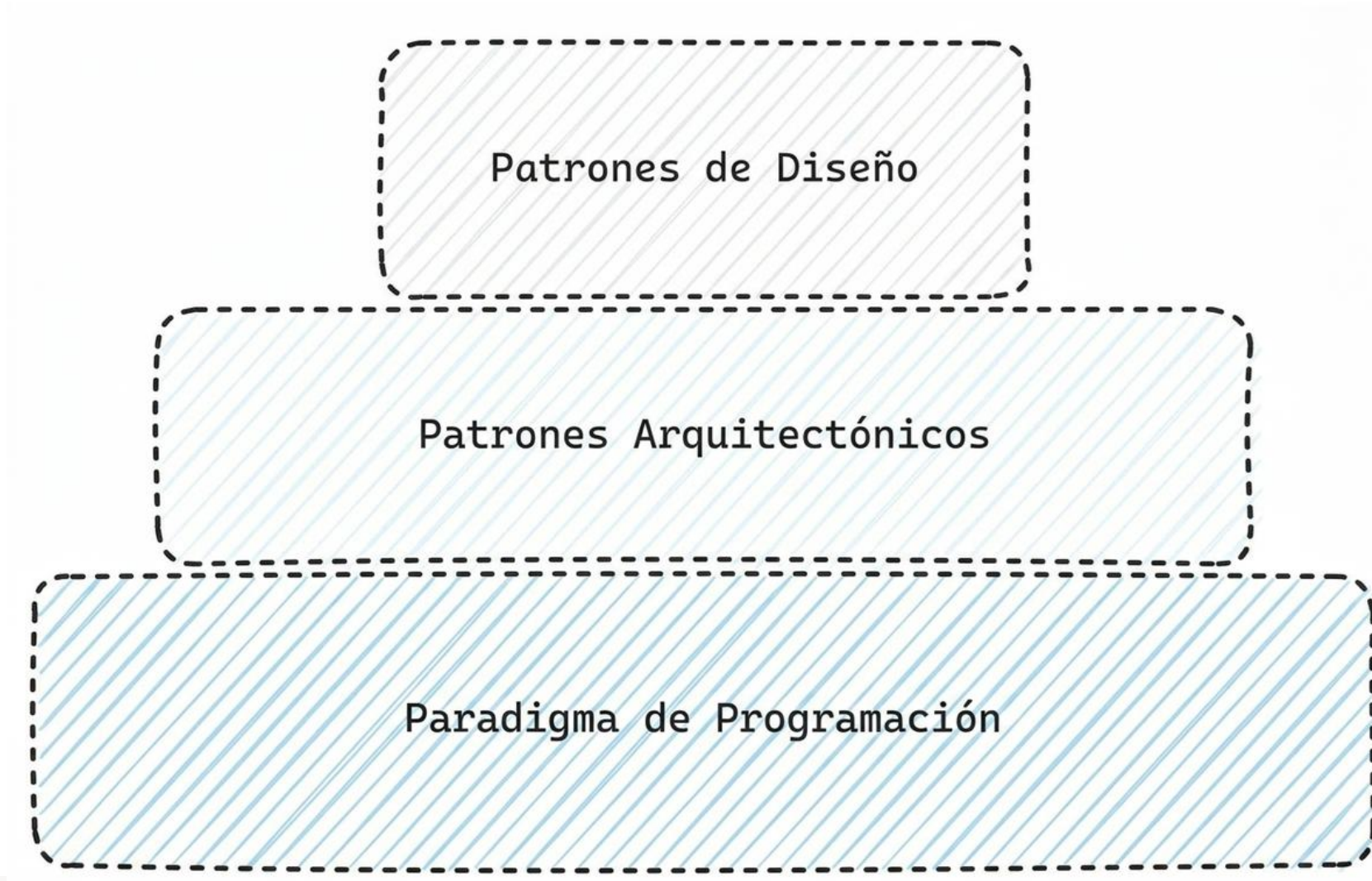
Prohibida la reproducción, total o parcial, sin autorización.

Ley nº 9610/98

Paradigma en la Programación



El Paradigma es la Base de la Solución



Paradigma Imperativo (Década de los 50)

- **Contexto:** El paradigma imperativo fue uno de los primeros estilos de programación adoptados, en una época en que el foco era controlar directamente los recursos de la computadora.
- **Lenguajes iniciales:** Assembly, Fortran.
- **Características:** Comandos secuenciales, uso de variables para almacenar estado y control explícito del flujo de ejecución.
- **Impacto:** Este paradigma sirvió como la base para la programación estructurada y ofrece control directo sobre el hardware.

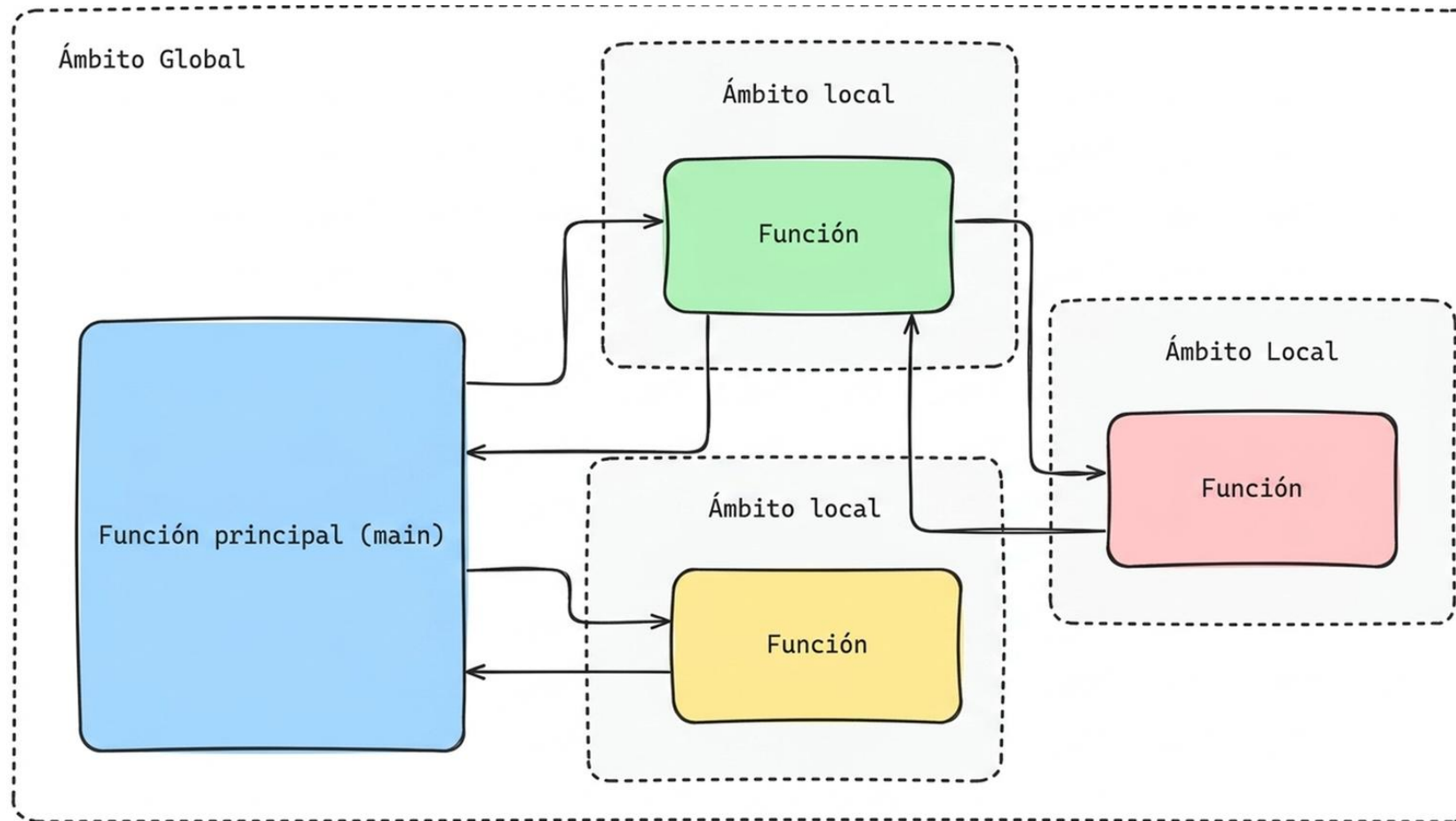
Paradigma Funcional (Década de 1950)

- **Contexto:** Basado en la teoría matemática de las funciones, este paradigma creció con la popularización de lenguajes específicos.
- **Lenguajes:** Lisp, Haskell.
- **Características:** El paradigma funcional evita estados mutables y efectos colaterales, priorizando funciones puras e inmutabilidad.
- **Impacto:** Ofrece ventajas en programación concurrente y procesamiento de datos, siendo muy utilizado en problemas de matemáticas y computación científica, así como en frameworks modernos para manipulación de datos y concurrencia.

Paradigma Procedural (Década de 1960)

- **Contexto:** El paradigma procedural surgió como una evolución natural del imperativo, al introducir estructuras que organizan el código en procedimientos o subrutinas. Esto ayudó a hacer el código más modular y reutilizable.
- **Lenguajes:** C, Fortran, COBOL, Pascal.
- **Características:** Organización del código en procedimientos o funciones que pueden ser llamados para realizar tareas específicas. El programa se construye como una secuencia de llamadas a estos procedimientos, lo que facilita el flujo controlado de ejecución.
- **Ventajas:** Mejor modularidad en comparación con el imperativo puro, permitiendo la reutilización de código y la reducción de redundancia.
- **Ejemplos de uso:** Aplicaciones científicas, ingeniería y sistemas empresariales que requieren procesos repetitivos y organizados.

Paradigma Procedural (Década de 1960)



Paradigma Estruturado (Década de los 60)

- **Contexto:** Conforme los programas se volvieron más complejos, surgió la necesidad de crear una estructura más organizada para el código.
- **Lenguajes:** C, Pascal.
- **Características:** Introducción de estructuras de control (como bucles y condicionales), subrutinas y funciones, facilitando el mantenimiento y la legibilidad del código.
- **Impacto:** El paradigma estructurado abrió camino para el desarrollo de programas más organizados, reduciendo la "codificación espagueti" (código desorganizado y de difícil lectura).

Paradigma Estruturado (Década de los 60)

Code

```
goto label_1;
```

```
statement_1;  
statement_2;
```

```
label_1:
```

```
statement_3;  
statement_4;
```

```
goto label_3;
```

```
statement_5
```

```
label_2;
```

Code

```
goto label_4;
```

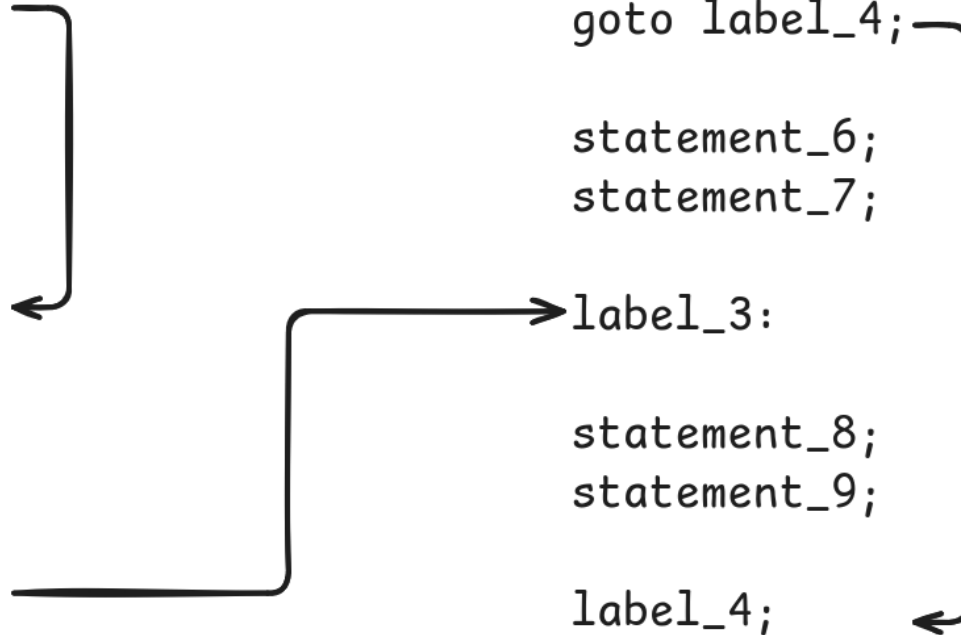
```
statement_6;  
statement_7;
```

```
label_3:
```

```
statement_8;  
statement_9;
```

```
label_4;
```

```
statement_10;  
statement_11;
```



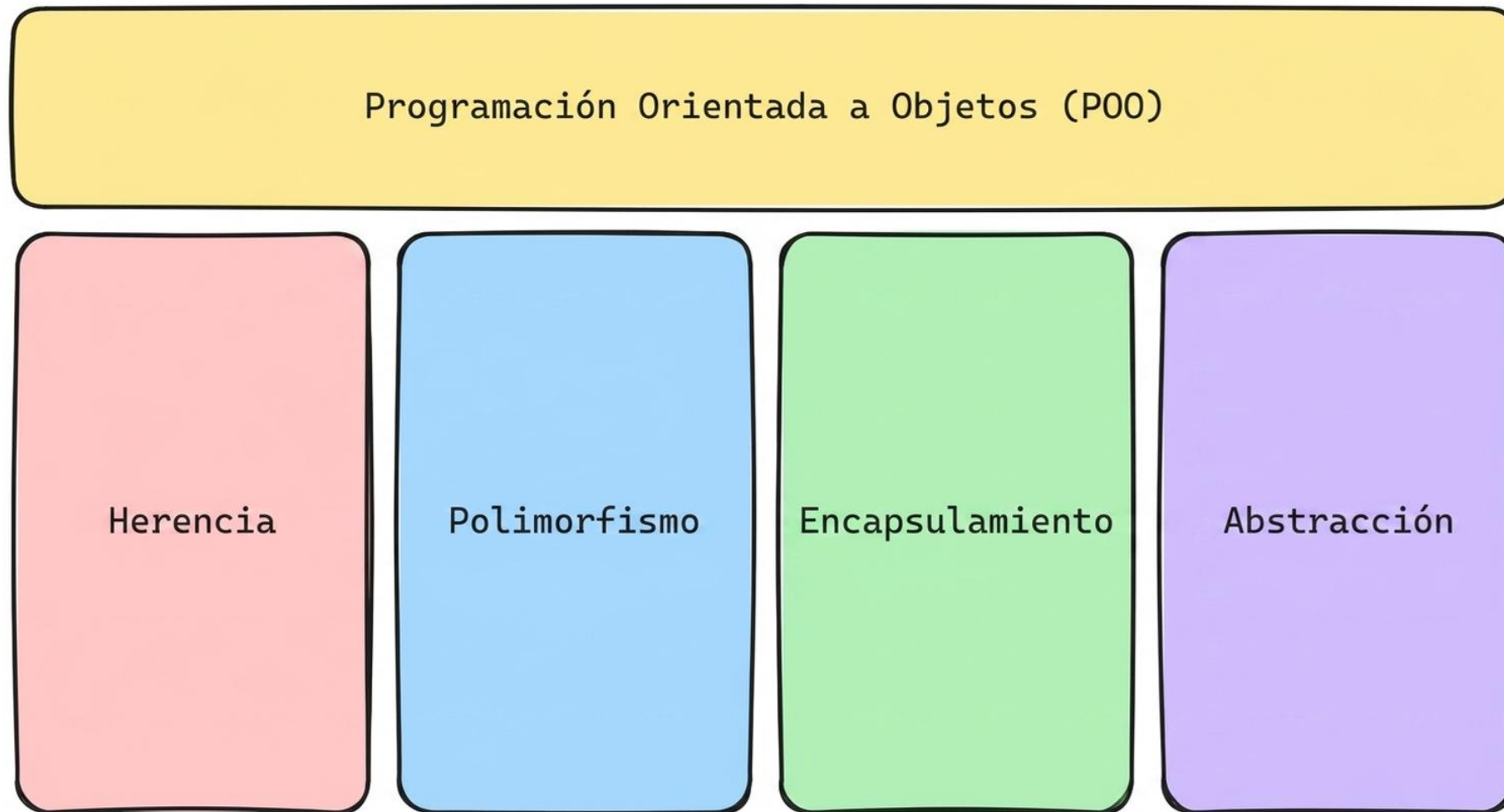
Paradigma Lógico (Década de 1970 y 1980)

- **Contexto:** Creado para aplicaciones de inteligencia artificial y resolución de problemas basados en lógica.
- **Lenguajes:** Prolog, Datalog.
- **Características:** Se enfoca en declarar relaciones y reglas que describen el problema, dejando para la computadora la búsqueda de soluciones lógicas.
- **Impacto:** Aunque no es ampliamente utilizado para aplicaciones comerciales, es esencial en áreas de IA, como lenguajes naturales e inferencia lógica.

Paradigma Orientado a Objetos (OO) (Década de los 80)

- **Contexto:** Surgió como respuesta a la necesidad de modularidad y reutilización de código para sistemas cada vez más complejos.
- **Lenguajes:** Java, C++.
- **Características:** Organiza el código en "objetos" que representan entidades del mundo real o conceptual, encapsulando datos y comportamientos.
- **Impacto:** Se convirtió en el paradigma dominante en desarrollo corporativo y juegos, promoviendo la creación de bibliotecas reutilizables y ayudando a gestionar la complejidad de sistemas grandes.

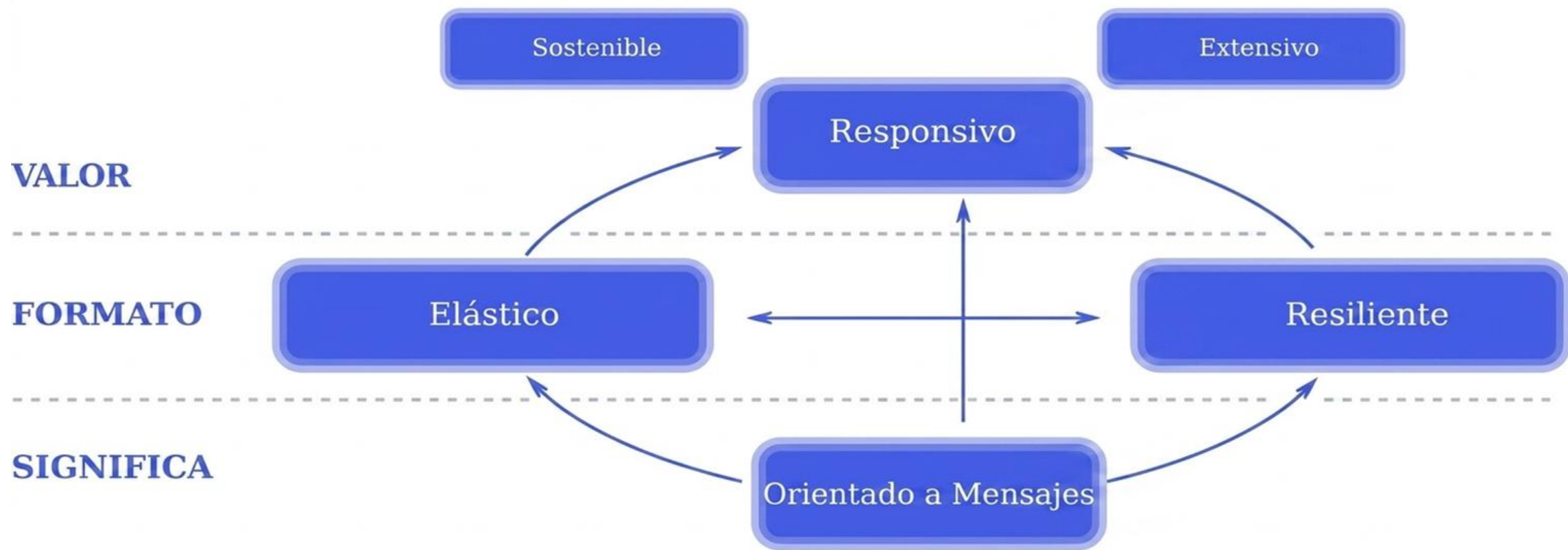
Paradigma Orientado a Objetos (OO) (Década de los 80)



Paradigma Reactivo (2000 en adelante)

- **Contexto:** Con el aumento de la demanda por sistemas en tiempo real y responsivos, el paradigma reactivo se volvió más relevante.
- **Herramientas y Tecnologías:** RxJS, Reactor, frameworks de programación reactiva.
- **Características:** Utiliza flujos de datos asíncronos y programación orientada a eventos, donde los cambios en una parte del sistema reaccionan automáticamente a los cambios en otra.
- **Impacto:** Esencial para aplicaciones modernas como dashboards en tiempo real, plataformas de streaming y sistemas distribuidos.

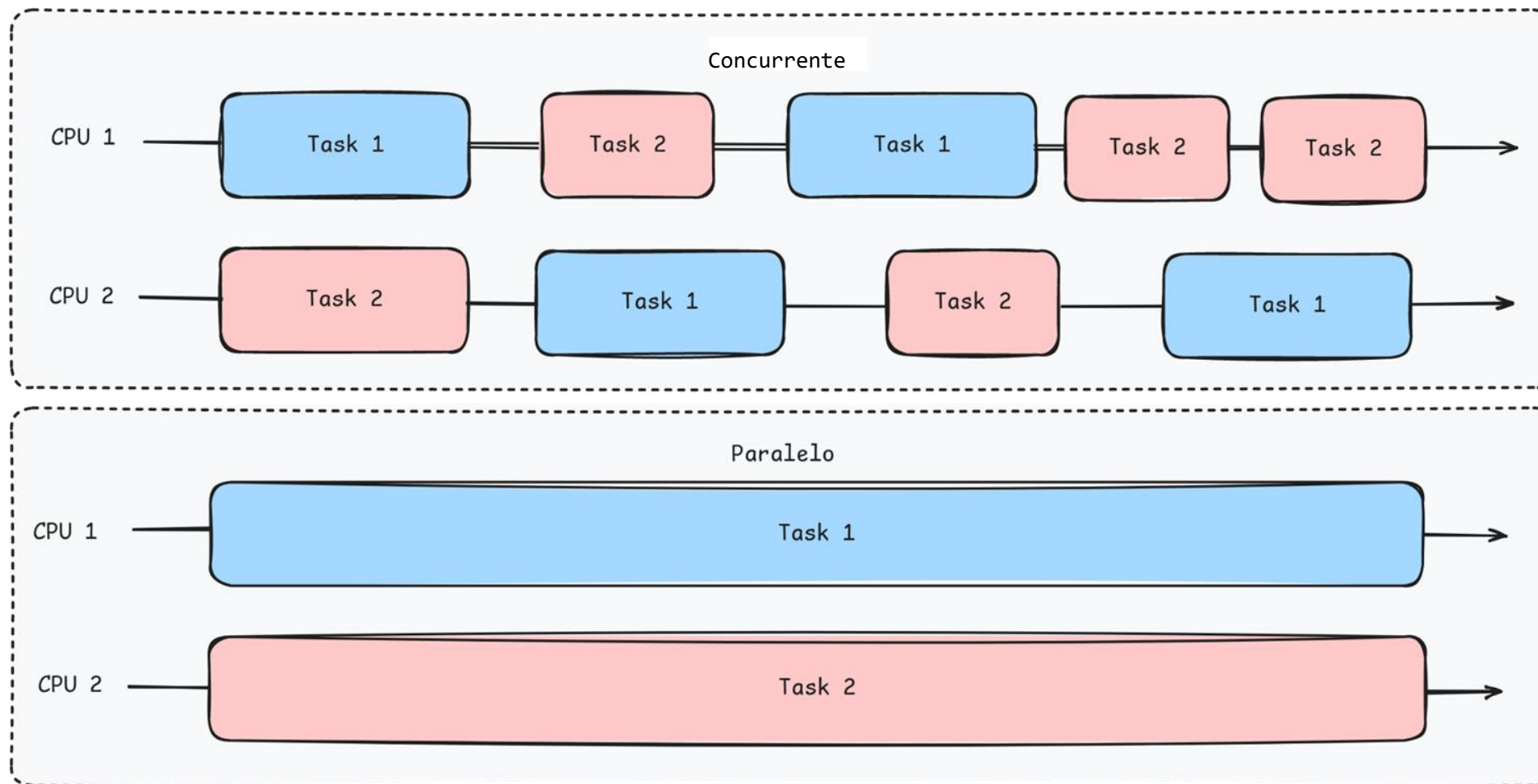
Paradigma Reactivo (2000 en adelante)



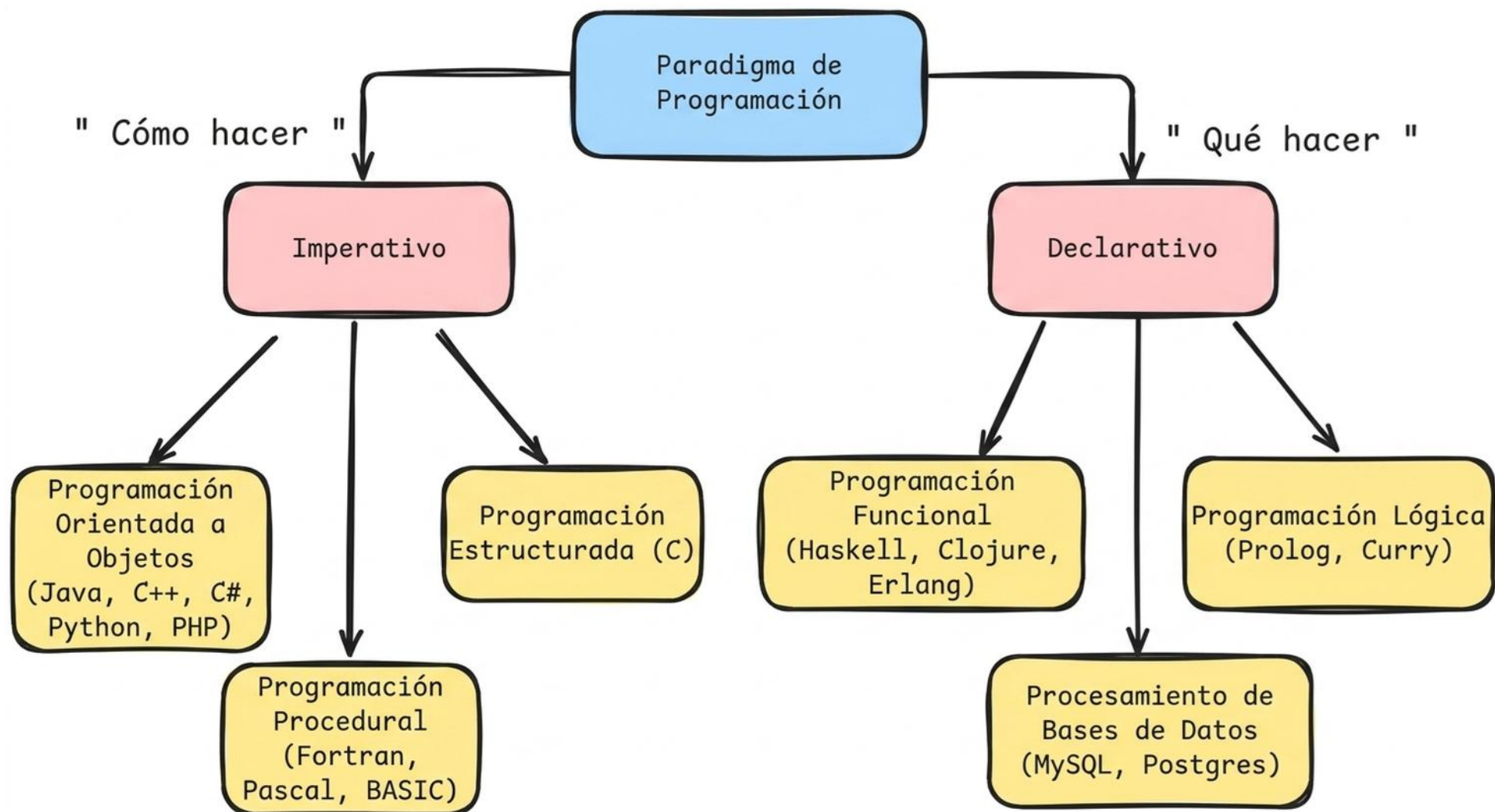
Paradigmas Concurrentes y Paralelos (a lo largo de las décadas)

- **Contexto:** Con la llegada de los procesadores multinúcleo, la programación concurrente y paralela se hizo necesaria para aprovechar mejor el hardware.
- **Herramientas y Tecnologías:** Threads, OpenMP, MPI, frameworks de actores como Akka.
- **Características:** Enfoque en la ejecución simultánea de múltiples tareas para aumentar el rendimiento y la eficiencia de aplicaciones que procesan grandes volúmenes de datos.
- **Impacto:** Esencial para áreas como la computación científica, simulaciones y big data, ayudando a crear programas más rápidos y capaces de procesar grandes volúmenes de datos en paralelo.

Paradigmas Concorrentes y Paralelos (a lo largo de las décadas)



Paradigma Imperativo y Declarativo



Paradigma Imperativo y Declarativo

Paradigma Imperativo	Paradigma Declarativo
Cómo hacer	Que hacer
Más verboso	Menos verboso
Variables mutables	Inmutabilidad
Comandos secuenciales	Conjunto de sentencias
Más explícito	Menos explícito
Mayor foco en el flujo de datos	Mayor foco en la lógica
Más legible y alterable	Más difícil de entender
Más difícil de escalar	Más fácil de escalar

Preparando una taza de café usando los paradigmas

Imperativo:

- "Juan, toma la cafetera. Pon agua hasta el nivel indicado. Abre el compartimento del café molido. Pon tres cucharadas de café molido. Cierra el compartimento. Coloca la cafetera en la base. Pulsa el botón de encendido. Espera hasta que el café esté listo. Toma una taza y pon el café en ella."

Declarativo:

- "Juan, haz una taza de café."

Ejemplo práctico de los paradigmas



```
# Lista de números
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Variable para almacenar la suma de los números pares
suma_pares = 0

# Bucle para iterar por la lista y sumar los pares
for numero in numeros:
    if numero % 2 == 0: # Verifica si el número es par
        suma_pares += numero # Añade el número par a la suma

print("Suma de los números pares:", suma_pares)
```

Ejemplo práctico de los paradigmas



```
# Lista de números
numeros = [1, 2, 3, 4, 5, 6, 7, 7, 8, 9, 10]

# Suma de los números pares usando funciones de orden superior
suma_pares = sum(filter(lambda x: x % 2 == 0, numeros))

print("Suma de los números pares:", suma_pares)
```

Ejemplo práctico de los paradigmas



```
SELECT nombre, edad  
FROM clientes  
WHERE ciudad = 'São Paulo' AND edad > 30;
```

Ejemplo práctico de los paradigmas



```
class Libro:
    def __init__(self, titulo, autor, ano_publicacion):
        self.titulo = titulo
        self.autor = autor
        self.ano_publicacion = ano_publicacion
        self.disponible = True

    def prestar(self):
        if self.disponible:
            self.disponible = False
            print(f'El libro "{self.titulo}" fue prestado.')

        self.disponible = True
        print(f'El libro "{self.titulo}" fue devuelto.')
```

Ejemplo práctico de los paradigmas

```
class Biblioteca:
    def __init__(self):
        self.libros = []

    def adicionar_libro(self, libro):
        self.libros.append(libro)
        print(f'El libro "{libro.titulo}" fue añadido a la biblioteca.')

    def listar_libros(self):
        print("\nLista de libros en la biblioteca:")
        for libro in self.libros:
            print(libro)

    def prestar_libro(self, titulo):
        for libro in self.libros:
            if libro.titulo == titulo:
                libro.prestar()
                return
        print(f'El libro "{titulo}" no fue encontrado en la biblioteca.')

    def devolver_libro(self, titulo):
        for libro in self.libros:
            if libro.titulo == titulo:
                libro.devolver()
                return
```

Ejemplo práctico de los paradigmas

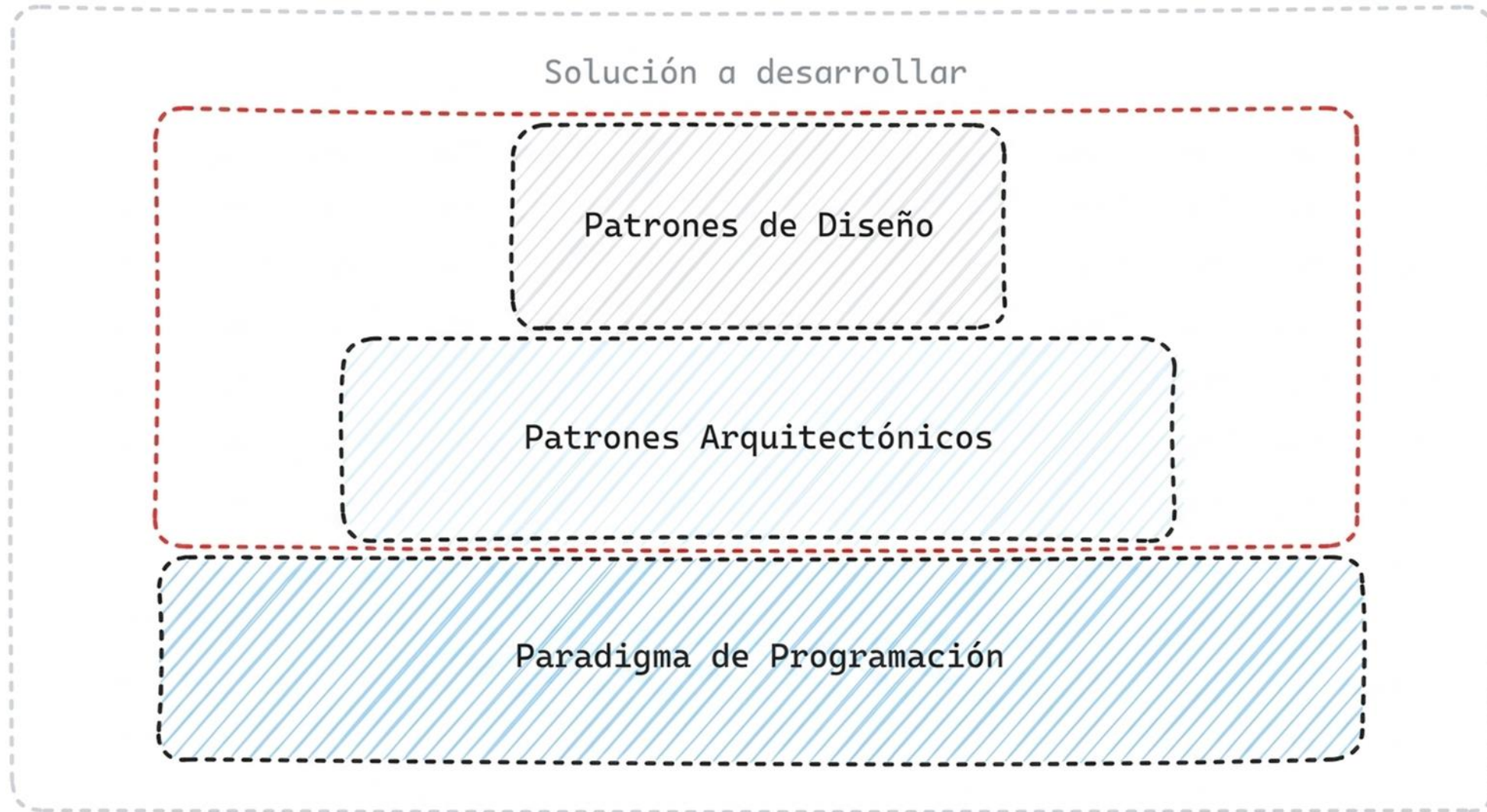


```
# Creando algunos libros
libro1 = Libro("1984", "George Orwell", 1949)
libro2 = Libro("El Señor de los anillos", "J.R.R. Tolkien", 1954)
libro3 = Libro("El Código Da Vinci", "Dan Brown", 2003)

# Añadiendo libros a la biblioteca
biblioteca.adicionar_libro(libro1)
biblioteca.adicionar_libro(libro2)
biblioteca.adicionar_libro(libro3)

# Listando libros en la biblioteca
biblioteca.listar_libros() # [libro1, libro2, libro3]
# Prestando un libro
biblioteca.prestar_libro("1984")
biblioteca.listar_libros() # [libro2, libro3]
# Devolviendo un libro
biblioteca.devolver_libro("1984")
biblioteca.listar_libros() # [libro1, libro2, libro3]
```

Eligiendo el Paradigma Adecuado



¡Gracias!

Prof. Helder Prado Santos | [linkedin.com/in/helderprado](https://www.linkedin.com/in/helderprado)

MBA **USP**
ESALQ